Node.js — Day 11

The path Module: dirname(), extname(), and Practical Backend Use

Learning goal: Finish the basic path toolkit and understand how path methods can be used together in a real backend file-handling scenario.

> **Previous knowledge:** path.join() and path.basename() were covered earlier. Today adds path.dirname() and path.extname().

1. path.dirname()

Purpose: Get the directory portion of a path.

```
const path = require("path");

const filePath = "/users/bhavana/documents/report.pdf";

console.log(path.dirname(filePath));
```

Output:

```
/users/bhavana/documents
```

2. path.extname()

Purpose: Get the file extension, including the leading dot.

```
const path = require("path");

const filePath = "/users/bhavana/report.pdf";

console.log(path.extname(filePath));
```

Output:

```
.pdf
```

Examples: .jpg, .csv, .js, .pdf.

3. The Four Basic path Methods

```
path.join()      → BUILD a path
path.dirname()   → GET the directory
path.basename()  → GET the filename
path.extname()   → GET the extension
```

Together these methods let you build and inspect a path string.

4. Practical Backend Example

```
const path = require("path");

const filePath = path.join(
  "uploads",
  "users",
  "bhavana",
  "profile.jpg"
);

const directory = path.dirname(filePath);
```

```
const fileName = path.basename(filePath);
const extension = path.extname(filePath);

console.log(directory);
console.log(fileName);
console.log(extension);
```

Output:

```
uploads/users/bhavana
profile.jpg
.jpg
```

A practical backend flow can be: user uploads a file → Node.js receives file information → build a path → work with the file/storage system → save relevant file information in the database.

> **Important:** The path module only works with path strings. It does not create, read, upload, or delete the actual file.

5. path vs fs

```
path → build and inspect paths
fs   → perform file-system operations
```

path.join() → build a path

path.basename() → get filename

path.dirname() → get directory

path.extname() → get extension

fs.readFile() → read a file

fs.writeFile() → write/replace a file

fs.unlink() → delete a file

Simple distinction: **path describes where something is; fs performs operations on files/directories.**

6. Important Rules

• dirname() returns the directory part of a path.

• basename() returns the final part, usually the filename.

• extname() returns the extension including the dot.

• join() combines path pieces into a proper path.

• The path module works with path strings; it does not perform file I/O itself.

• Use path utilities instead of manually assuming a path separator.

7. Quick Revision

```
/uploads/images/profile.png

path.dirname()  → /uploads/images
path.basename() → profile.png
path.extname()  → .png
```

To build a path from pieces:

```
path.join("uploads", "images", "profile.png")
```

8. Practice Completed

Question 1 ■ dirname() → /uploads/images; basename() → profile.png; extname() → .png

Question 2 ■ path.join() plus dirname(), basename(), and extname() gives the directory, filename, and .pdf extension.

Question 3 ■ All four methods were correctly combined for an uploaded profile.jpg path.

Score: 3 / 3 correct.

9. Day 11 Quick Recap

join() → BUILD

dirname() → DIRECTORY

basename() → FILE NAME

extname() → EXTENSION

path → works with path strings

fs → performs file-system operations

Key Takeaway

The path module helps Node.js safely construct and inspect paths. In real backend applications, path is often used together with fs or storage systems when handling uploaded, generated, or stored files.